

Desde las bases hasta Stable Diffusion

Proyecto INVERSO

Seminario bi-semanal

Adrian Maiza Coupin

Rubén Pascual Casas

upna

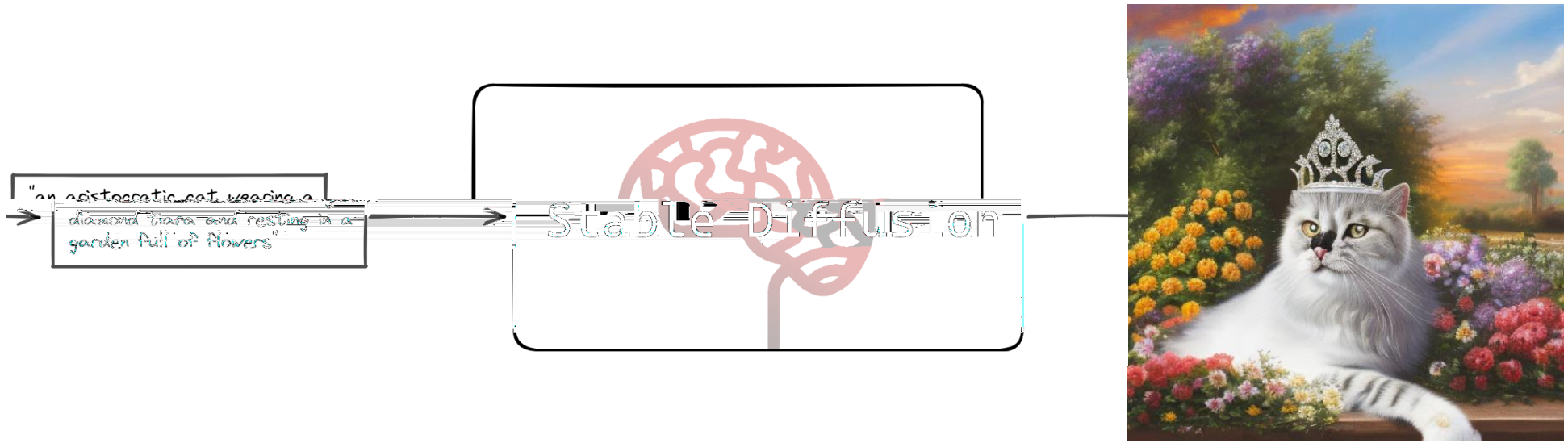
Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

Introducción

¿Qué hace Stable Diffusion?

Stable Diffusion es, en resumidas cuentas, un modelo **text-to-image**. Es decir, dado un texto de entrada o *"prompt"*, el modelo es capaz de generar una imagen que refleja el contenido de dicho texto.

Por ejemplo, si le damos como entrada *"an aristocratic cat wearing a diamond tiara and resting in a garden full of flowers"*, el modelo debería ser capaz de mostrarnos una imagen de un gato con una tiara de diamantes descansando en un jardín lleno de flores. Aunque el *prompt* es bastante extravagante, vamos a ver que nos devuelve.



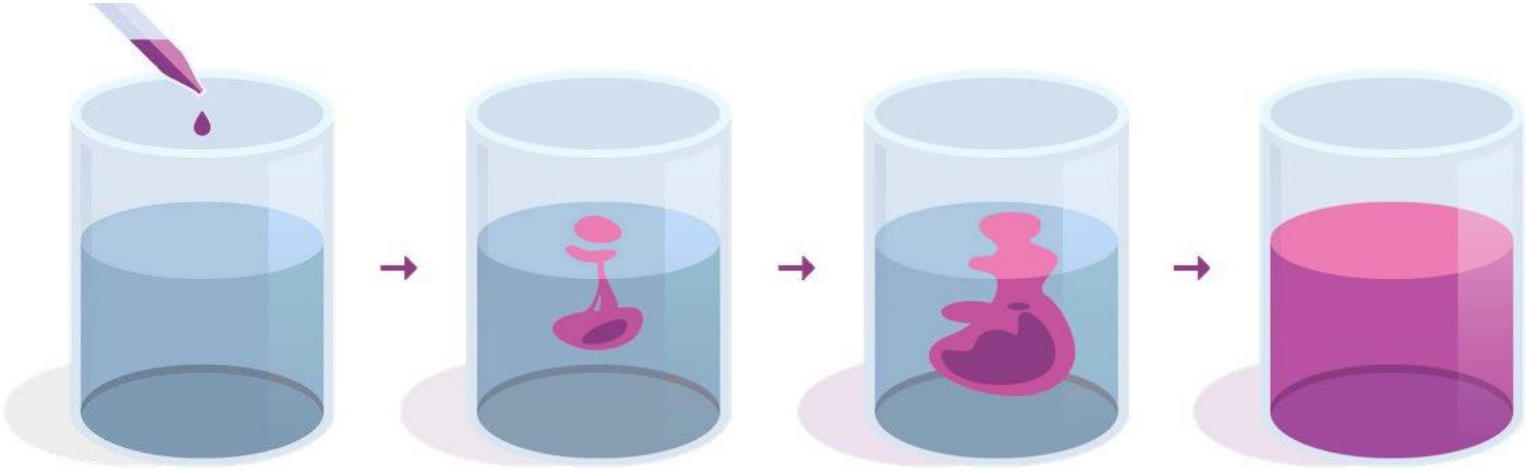
Índice

- Modelos de Difusión
  - Denoising Diffusion Probabilistic Models: definición y un poco de notación.
  - Cómo se entrenan estos modelos
  - Cómo obtenemos nuevas imágenes
- Stable Diffusion
  - Qué tiene de nuevo
  - Qué componentes tiene y cómo funcionan
  - Cómo se entrena
  - Cómo obtenemos nuevas imágenes
- Expandiendo el conocimiento de Stable Diffusion
  - Textual Inversión
  - Dreambooth
  - Custom Diffusion
  - LoRA

¿Qué son los modelos de difusión?

Stable Diffusion es un tipo de **modelo de difusión** dentro de los **modelos generativos**. Sabemos qué es un modelo de generativo, pero, ¿Qué significa exactamente el término ***difusión***?

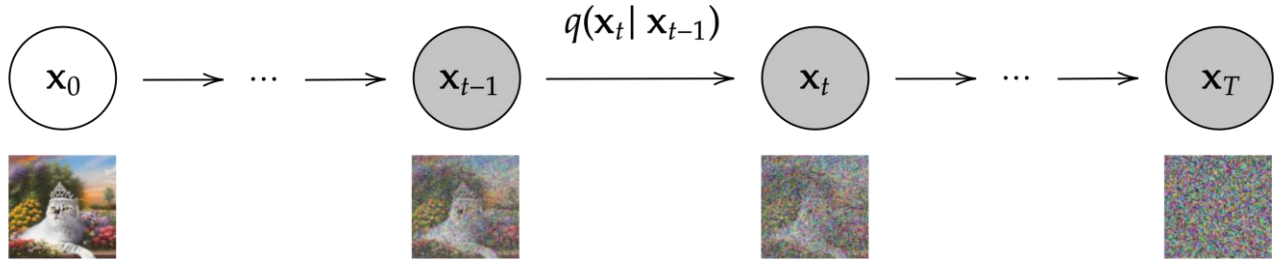
El proceso de difusión utilizado en este tipo de modelos está fuertemente inspirado en la difusión en física, concretamente en la "termodinámica del no-equilibrio". Ya que el concepto es bastante complejo y abstracto, veamos un ejemplo:



¿Cómo definimos el proceso de difusión para un modelo generativo?

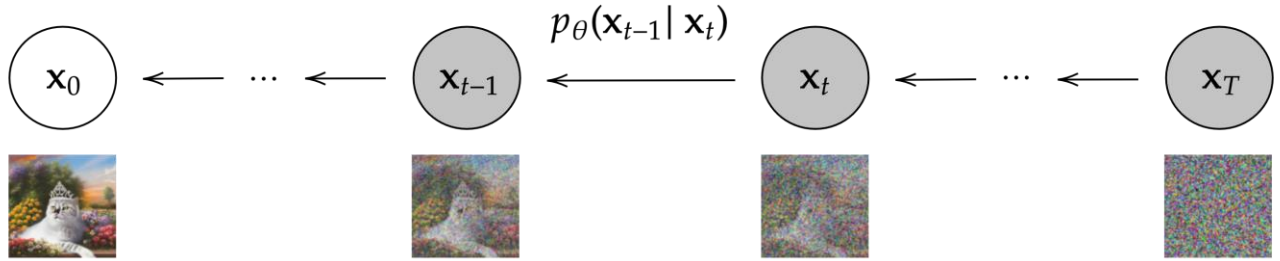
Debemos definir dos procesos: el proceso de **difusión hacia delante** y el de **difusión inversa**.

Difusión hacia delante



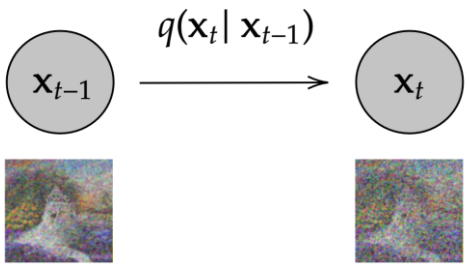
HABEMUS MODEL

Difusión Inversa

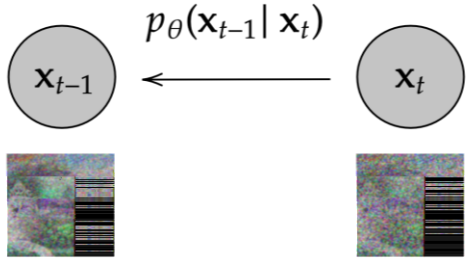


Denoising Diffusion Probabilistic Models (DDPM)

Vamos a ver un poco en detalle las matemáticas detrás de esta técnica.



- Se conoce como el *kernel de difusión hacia delante*
- Define la probabilidad de la imagen $\boldsymbol{x}_t$ dada la imagen $\boldsymbol{x}_{t-1}$.
- $q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}) = \mathcal{N}\left(\boldsymbol{x}_t; \sqrt{1-\beta_t}\boldsymbol{x}_{\mathrm{t}-1}, \sqrt{\beta_t}\boldsymbol{I}\right)$
- A *grosso modo*, devuelve la imagen $\boldsymbol{x}_t$ que es el resultado de añadir una imagen de ruido gaussiano a la imagen $\boldsymbol{x}_{t-1}$.



- Se conoce como el *kernel de difusión hacia atrás*
- Define la probabilidad de la imagen $\boldsymbol{x}_{t-1}$ dada la imagen $\boldsymbol{x}_t$.
- A *grosso modo*, es el resultado de quitarle el ruido gaussiano a la imagen en el paso *t* para obtener la imagen $\boldsymbol{x}_{t-1}$
- $p_\theta(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t) = \mathcal{N}(\boldsymbol{x}_{t-1}; \boldsymbol{\mu}_\theta(\boldsymbol{x}_t, t), \boldsymbol{\Sigma}_\theta(\boldsymbol{x}_t, t))$

¿Para qué se usan las $_t$?

Los valores de las β_t se extraen de o

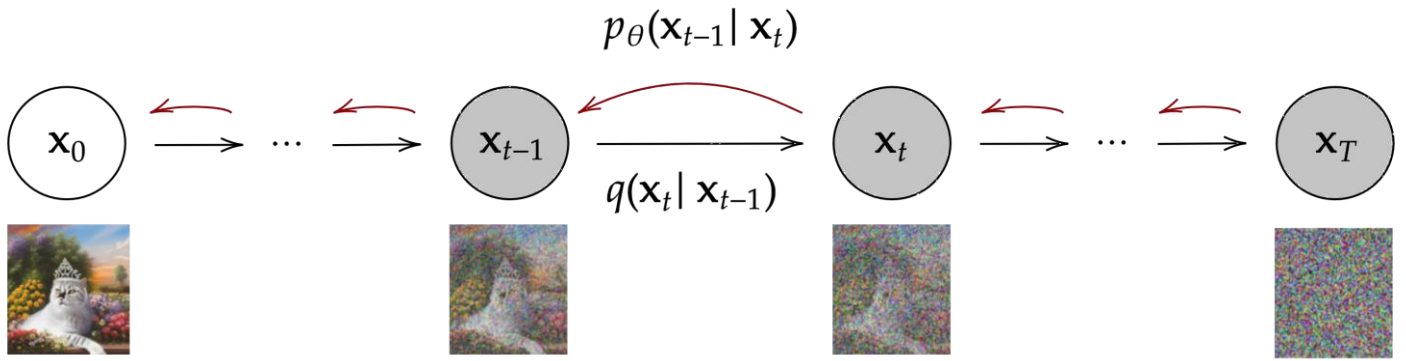
Pero, ¿Cómo entrenamos a este modelo?

Para poder invertir la difusión, necesitamos saber **qué ruido** se ha añadido a la imagen. Por lo tanto, podemos entrenar una **red neuronal** p con una arquitectura **U-Net** que prediga **cuál** ese **ruido añadido** a la imagen.

Una primera aproximación del entrenamiento sería la siguiente:

- Obtenemos un dataset de imágenes intermedias x_t haciendo uso del proceso de difusión hacia delante.
- Para cada imagen x_t predecimos el ruido que hay que quitarle para obtener la imagen x_{t-1} mediante la **U-Net**.
- Actualizamos la **U-Net** con el la diferencia entre el ruido predicho y el ruido original.

¡¡Esto plantea un problema de rendimiento!!



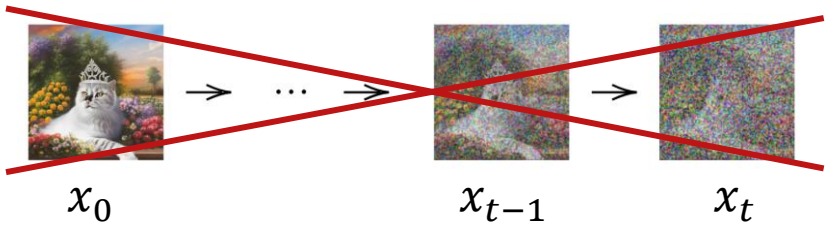
Podemos solucionarlo

Por eso, en vez de intentar optimizar todo el proceso para cada imagen, se **escoge un *t* aleatorio** y se intenta predecir **todo el ruido** aplicado a la **imagen original**. Pero, ¿Cómo obtenemos la imagen corrupta en el paso t sin todo ese proceso iterativo?

Gracias a las propiedades de las distribuciones gaussianas, podemos obtener el ruido del paso t combinando todos los ruidos anteriores:

$$q(x_t|\, x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_{t-1}, \sqrt{1-\bar{\alpha}_t}\mathrm{I}) \longrightarrow x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\,\mathcal{N}(0,\mathrm{I})$$

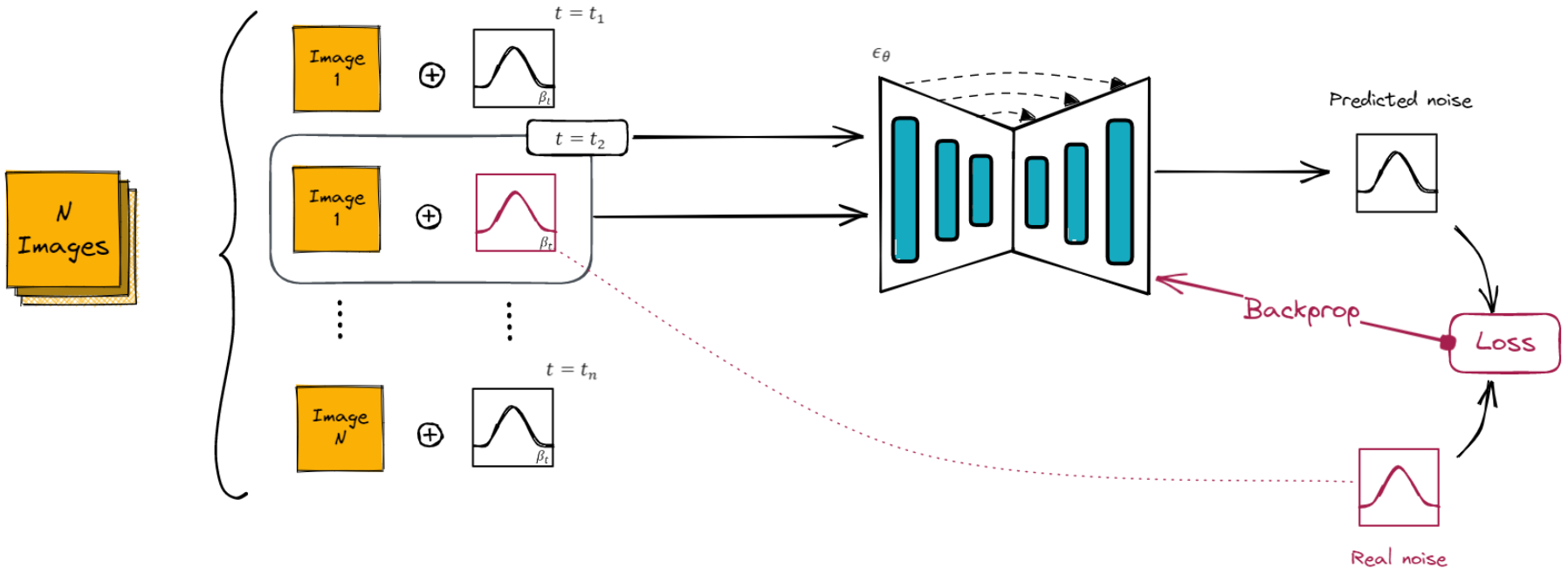
Donde $\bar{\alpha}_t = \prod_{i=1}^{t}\alpha_i\,, \alpha_t = 1-\beta_t$





¿Entonces cual es el método de entrenamiento?

El proceso es parecido al que hemos comentado antes. La diferencia es que, en este caso, la **U-Net** va a intentar predecir el ruido aplicado a x_0 que da la imagen x_t. Esta predicción la va a hacer con x_t y t como entrada. Una vez predicho el ruido, el error se calcula con el error cuadrático medio.

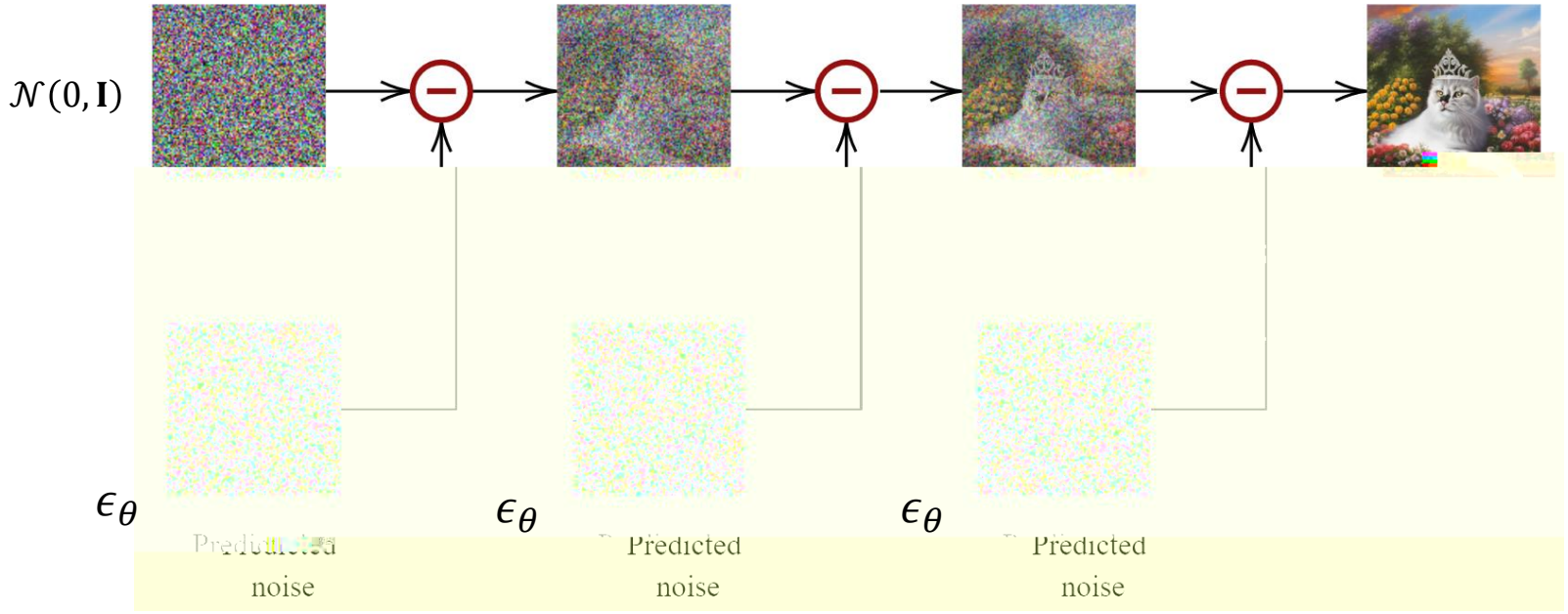


$$t\sim[1,T],\, x_0,\epsilon[\| \epsilon - \epsilon_\theta(x_t, t) \|]$$

Obteniendo nuevas imágenes

Una vez tenemos la **U-Net** entrenada, ahora la podemos utilizar para generar nuevas imágenes a partir de una distribución que conocemos, una distribución gaussiana.

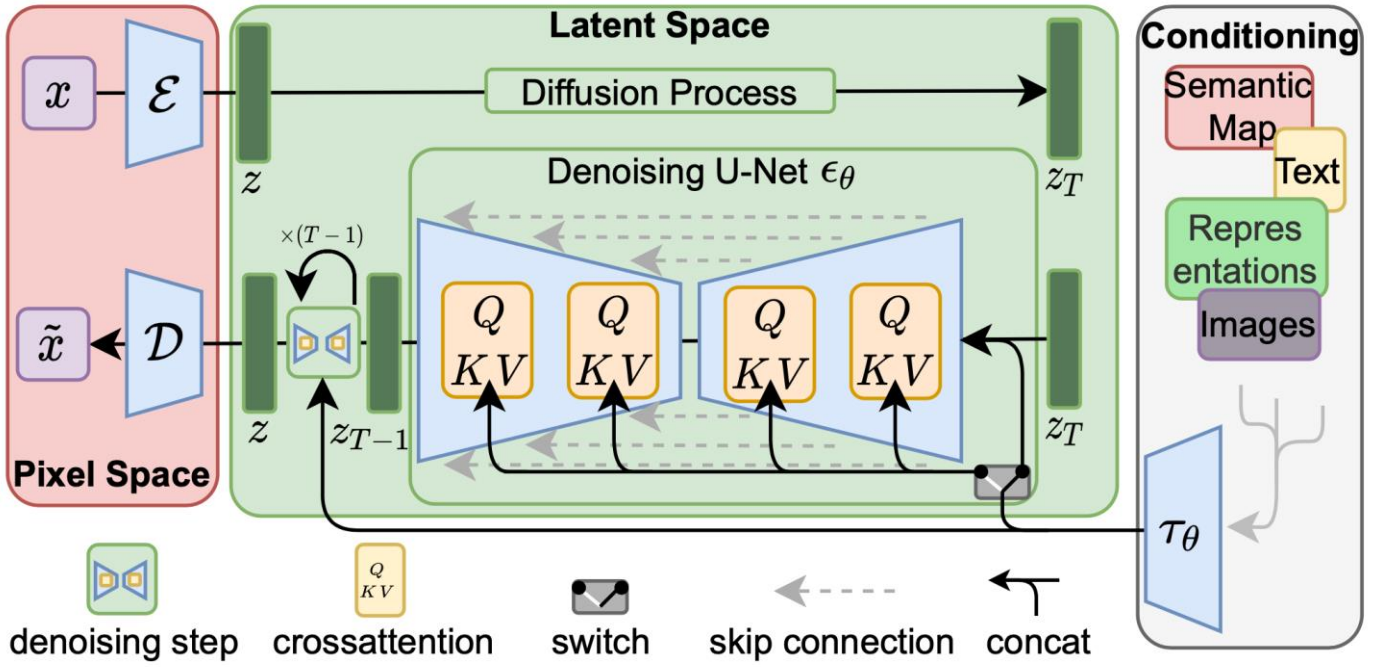
Para hacer esto, partimos de una **imagen de ruido aleatoria** sampleada de una distribución gaussiana de la forma $\mathcal{N}(0, \mathbf{I})$.



Stable Diffusion

¿De donde viene Stable Diffusion?

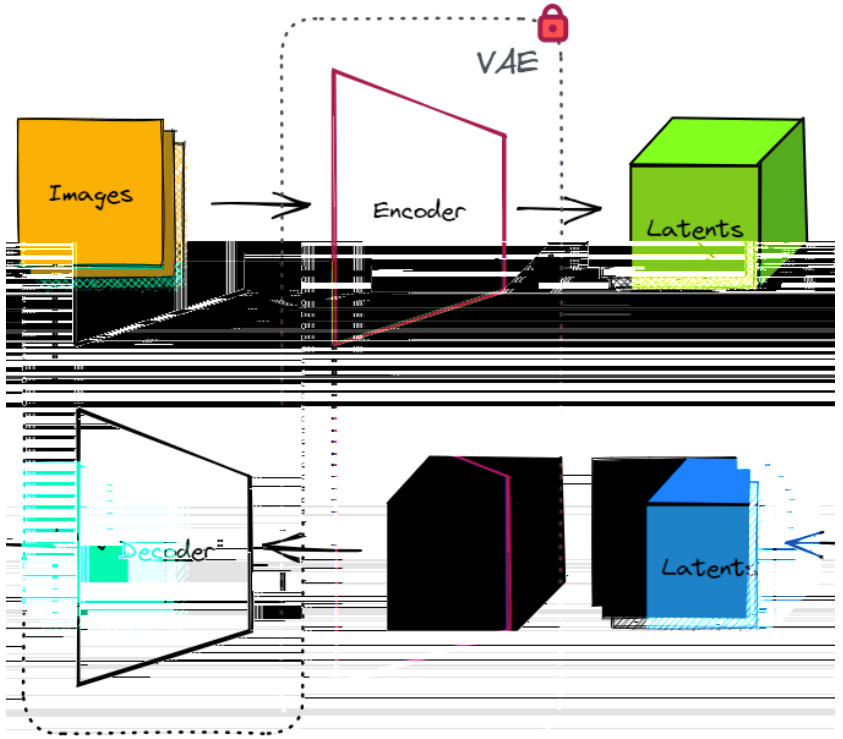
Stable Diffusion es una **implementación** y **mejora** del artículo "*High-Resolution Image Synthesis with Latent Diffusion Models*", donde se presentan los **Modelos de Difusión Latentes** (LDM). Este tipo de modelos están basados en los "*Denoising Diffusion Probabilistic Models*" añadiendo una transformación de los datos previos al proceso de difusión además de la condicionalidad de la salida.



Componentes de Stable Diffusion

Variational Auto-Encoder

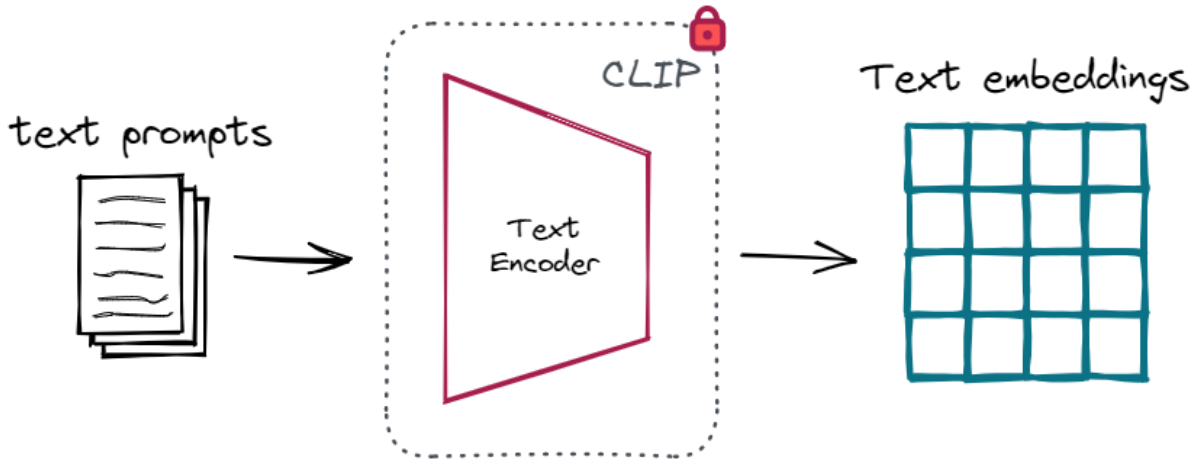
El **Auto-Encoder Variacional** (VAE), en este caso, es un modelo (**pre-entrenado** con imágenes) que **comprime** la información de los datos, transformándolos del espacio **de** los **pixeles a** un espacio de **latentes**. Esto se utiliza para reducir la dimensionalidad de los datos y así reducir el coste computacional del modelo completo.



Componentes de Stable Diffusion

Encoder de texto (CLIP)

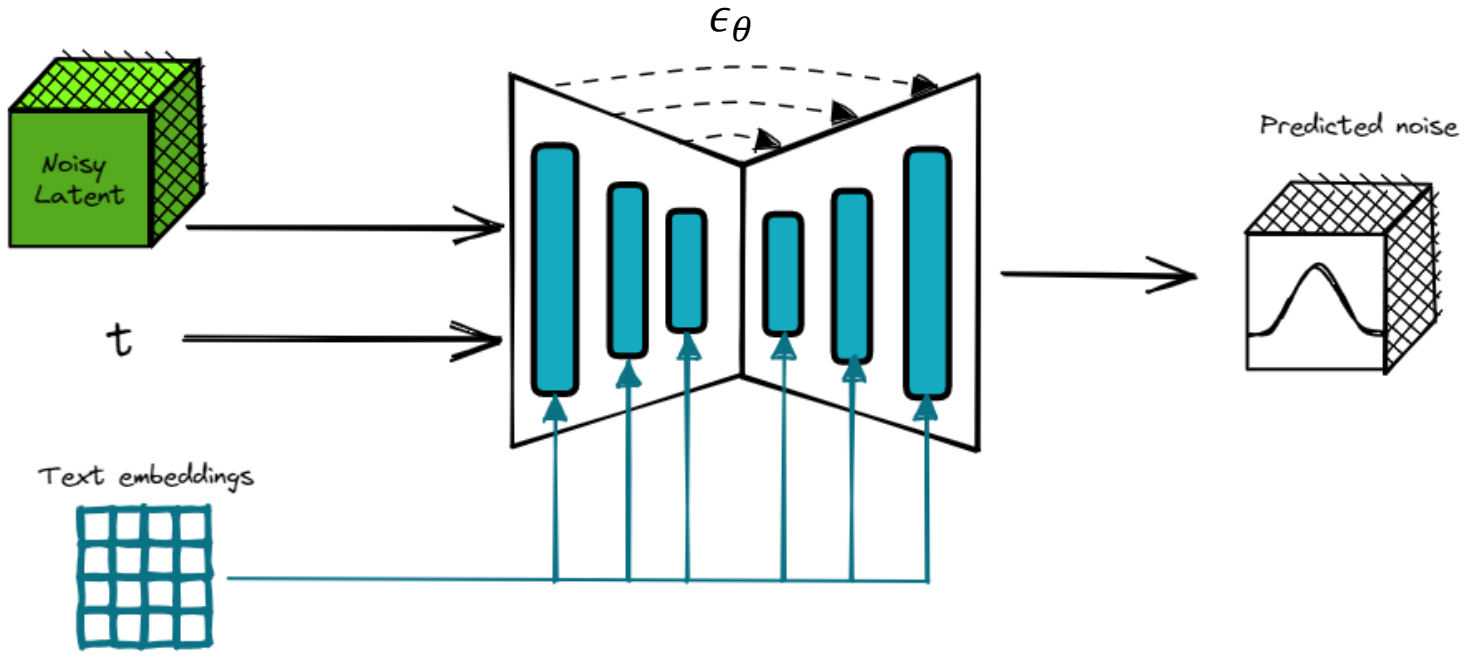
Los modelos de difusión permiten condicionar la generación de los datos. En este caso, Stable Diffusion utiliza **CLIP**, un modelo que trata de **alinear** los *embeddings* de las **imágenes** con los *embeddings* de sus respectivas **descripciones**. Para Stable Diffusion, el único componente importante de CLIP es el **encoder** que devuelve los ***embeddings* texto**.



Componentes de Stable Diffusion

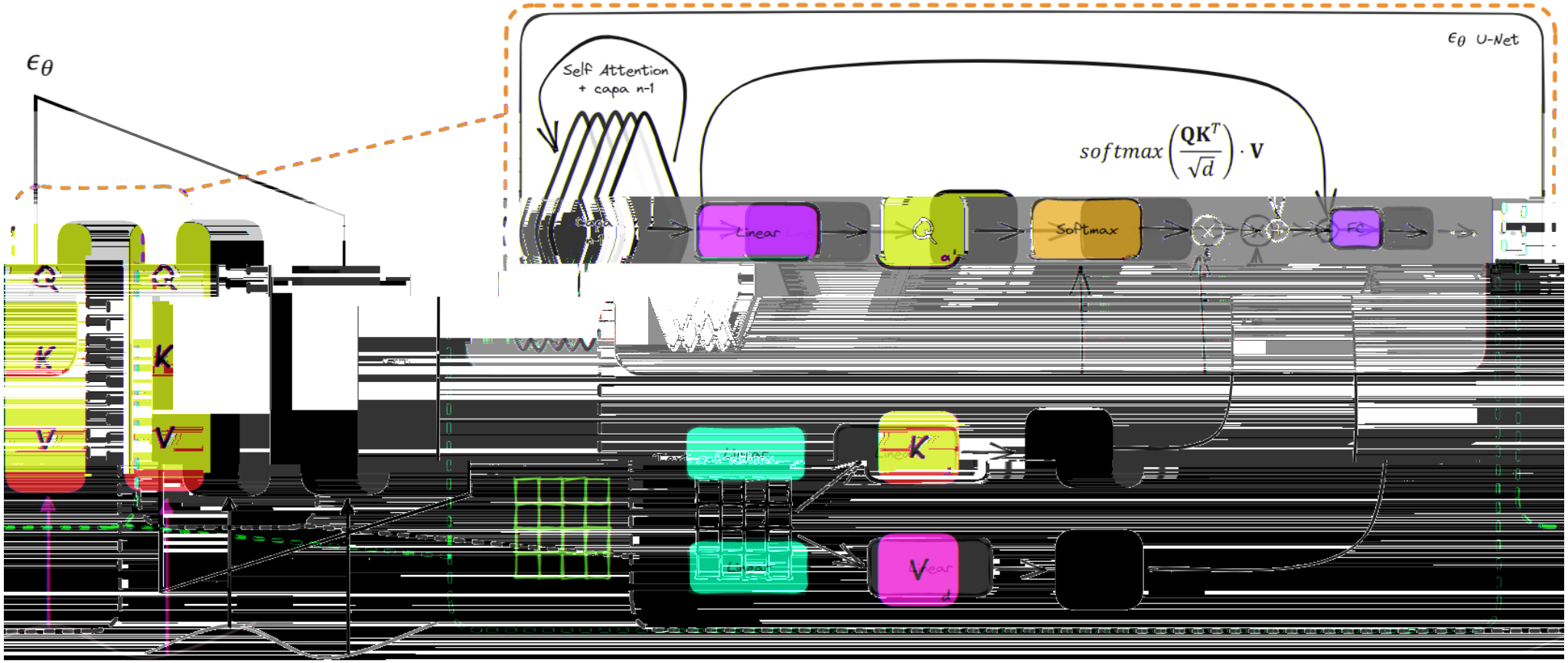
Denoising U-Net

El componente más importante de la arquitectura de Stable Diffusion es la U-Net condicionada ϵ_θ. Esta red tiene como objetivo predecir cual es el ruido aplicado a $\boldsymbol{x}_0$ que ha resultado en $\boldsymbol{x}_t$. Así, si restamos ese ruido a $\boldsymbol{x}_t$ deberíamos obtener $\boldsymbol{x}_0$, la imagen original.

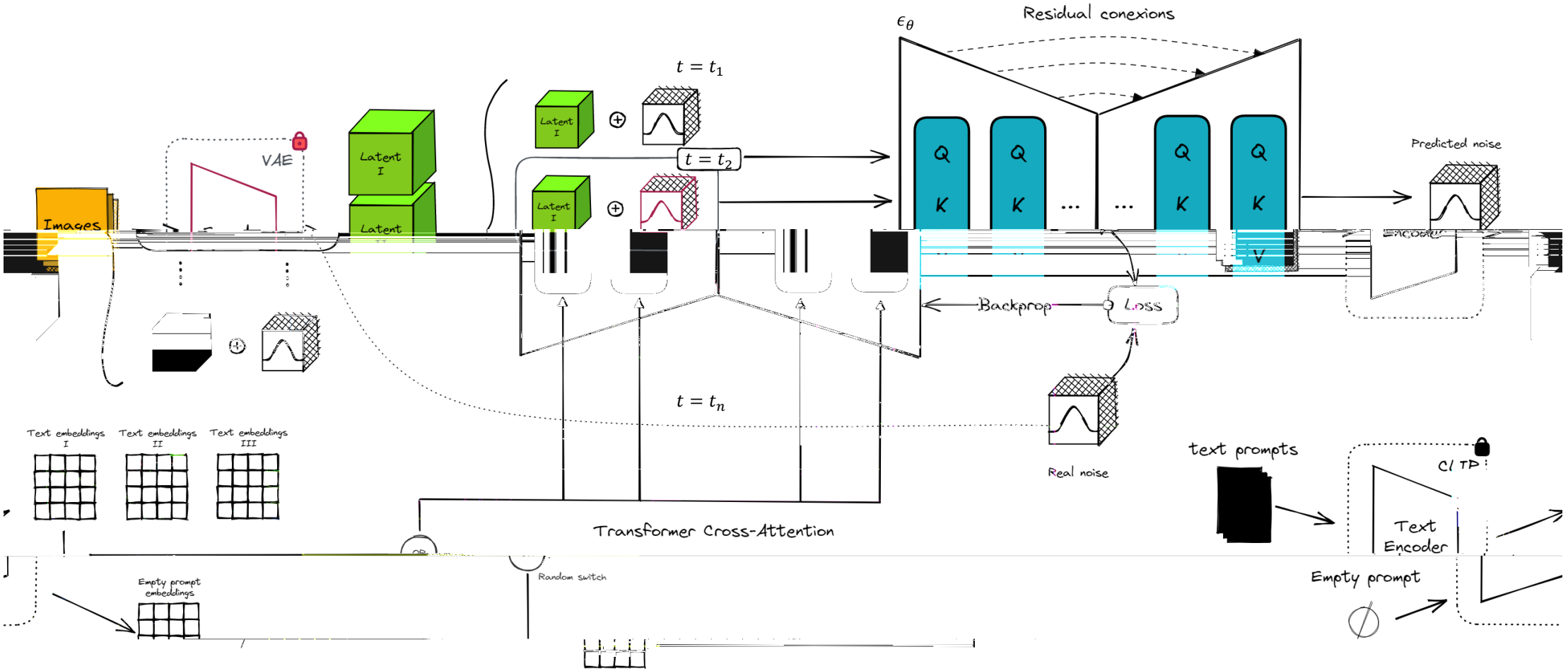


Componentes de Stable Diffusion

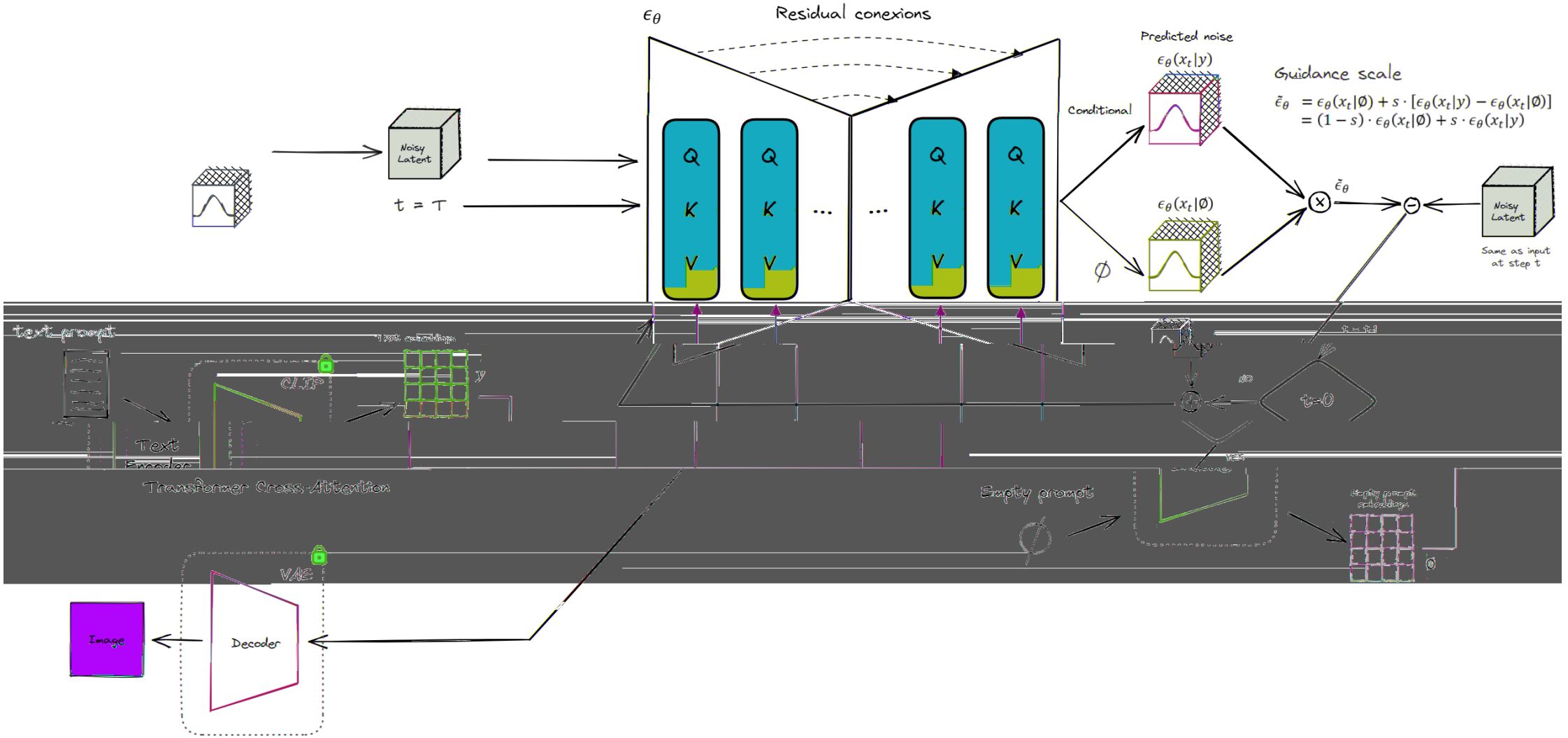
Capa transformers



Entrenamiento de Stable Diffusion



Inferencia en Stable Diffusion



Expandiendo el conocimiento de Stable Diffusion

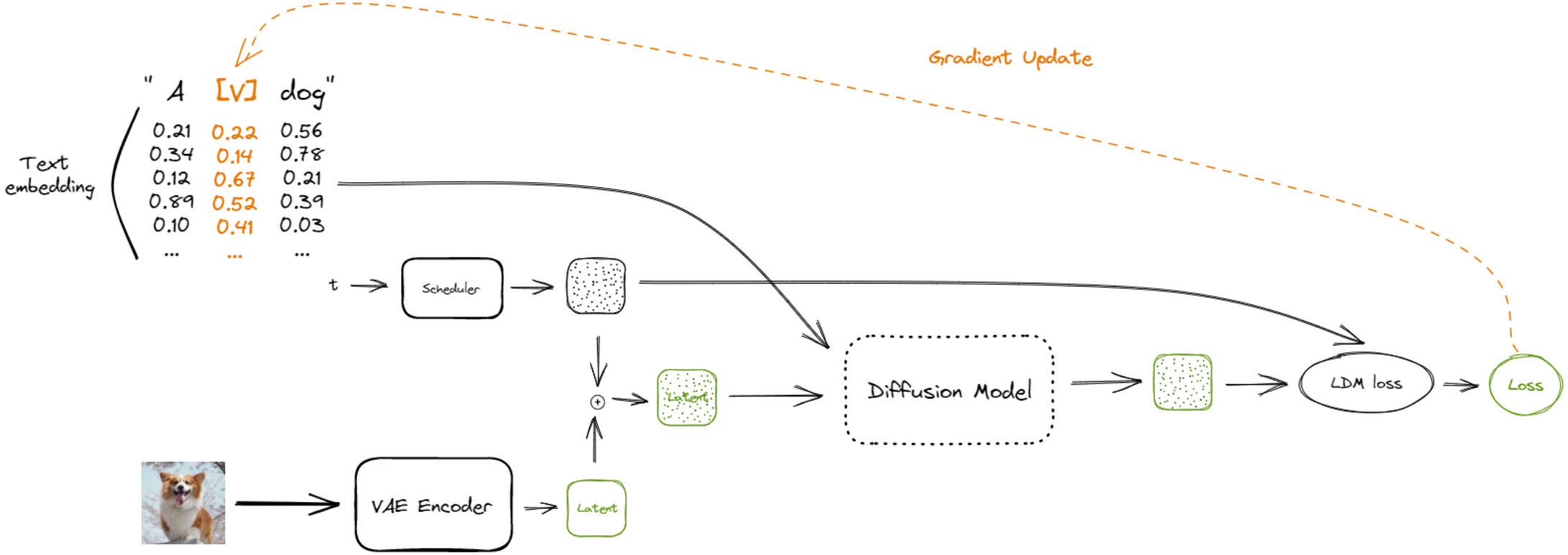
¿Que métodos existen a día de hoy?

Al ser un tema con mucho auge en los últimos meses, el conjunto de métodos para "reentrenar" el modelo de **Stable Diffusion**, haciendo que éste **genere datos de nuevos dominios**, es bastante amplio. La mayoría de estos métodos pretenden añadir nuevo conocimiento al modelo, expandiendo así la variedad de imágenes que puede generar. Es decir, le enseñan **nuevos conceptos** como "*fotos de mi gata Luna*", "*diseños de personajes con el estilo de nuestro dibujante*", *"edificios diseñados por mí"* o *"fotos de la tía Pili"* que éste, de base, **no entiende**.

Por ahora, identificamos ***cuatro*** métodos para conseguir esto:

Textual Inversion | **DreamBooth** | **Custom Diffusion** | **LoRA**

Textual Inversion



Dreambooth

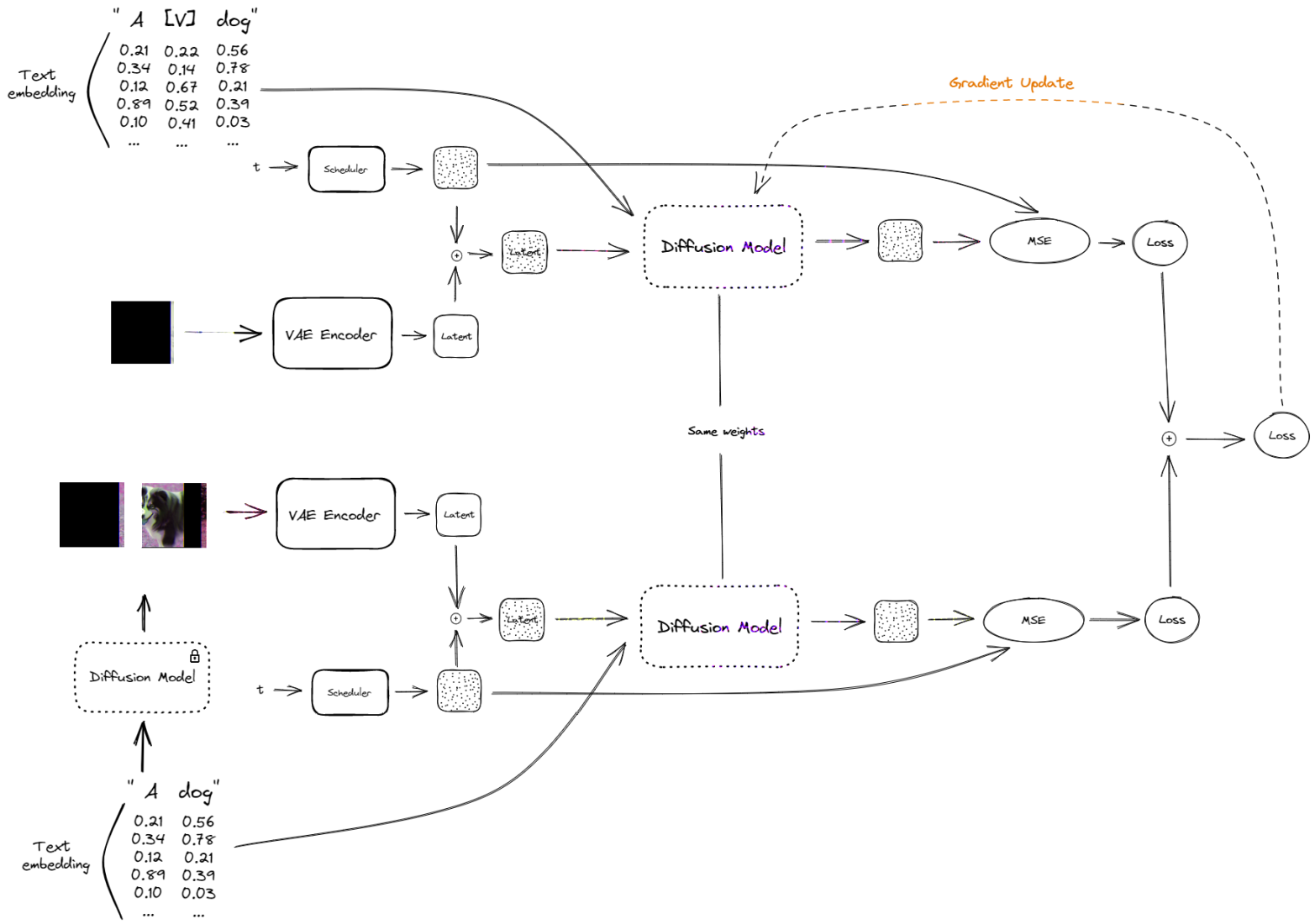
"A [identifier] [class noun]"

1) Utilizar una palabra existente como identificador (único/especial) ->
   Aumenta el tiempo de entrenamiento y disminuye el rendimiento

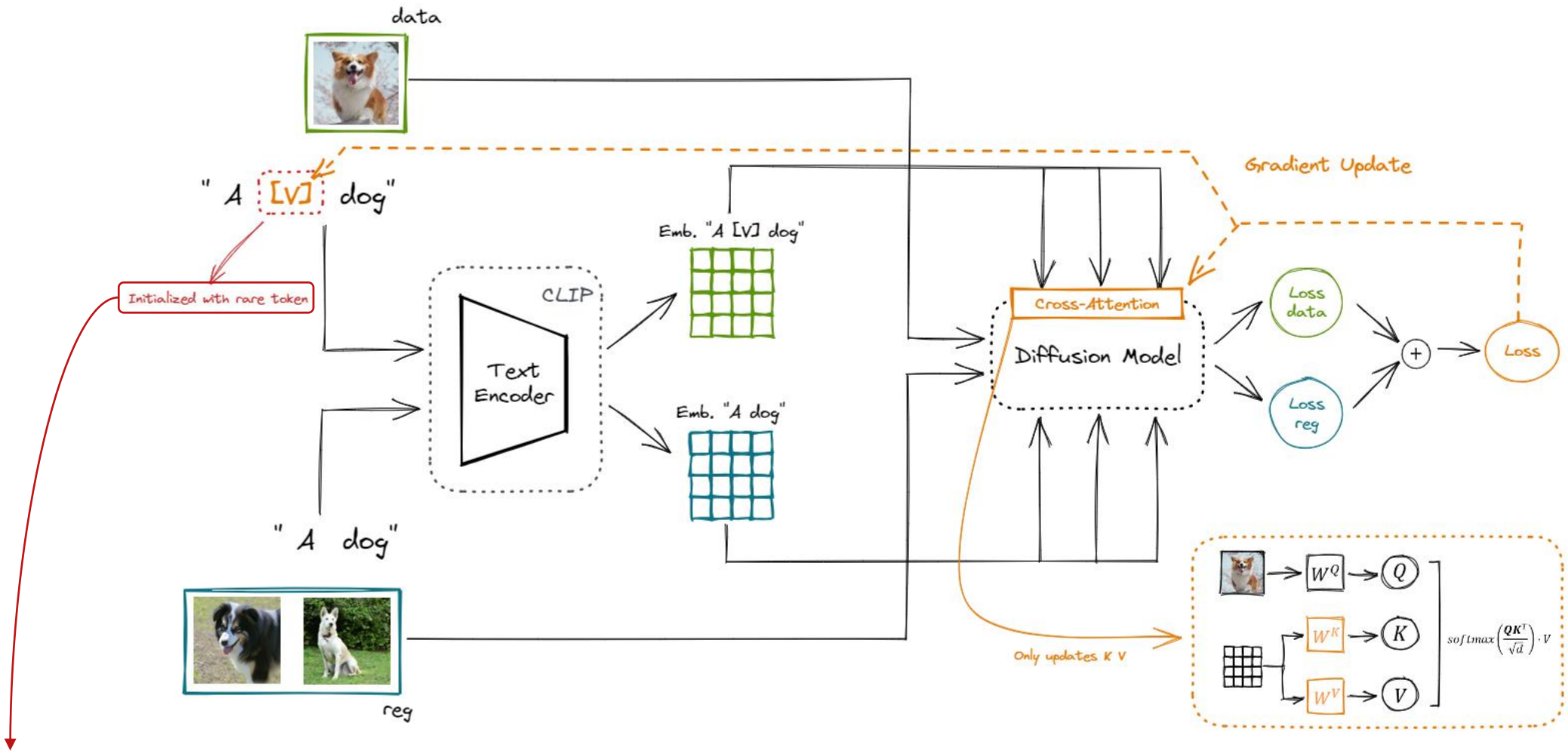
2) Generar un identificador raro mezclando letras/números aleatorios (xxy5st) ->
   El tokenizador podría tokenizar cada letra por separado

3) Encontrar un token raro en el vocabulario y luego invertirlos en el espacio de texto
   Al final se buscan palabras con longitud 3

Dreambooth


" A [V] dog"
Text embedding
0.21 0.22 0.56
0.34 0.14 0.78
0.12 0.67 0.21
0.89 0.52 0.39
0.10 0.41 0.03
t
Scheduler
Latent
Gradient Update
Diffusion Model
MSE
Loss
VAE Encoder
Latent
Same weights
Loss
VAE Encoder
Latent
Diffusion Model
Diffusion Model
MSE
Loss
t
Scheduler
" A dog"
Text embedding
0.21 0.56
0.34 0.78
0.12 0.21
0.89 0.39
0.10 0.03

Custom Diffusion



Conseguimos el "token raro" seleccionando un $V^* \in P_{200k}$, donde $P_{200k} \subseteq \text{LAION}_{400\text{M}}$.

Este V^* debe cumplir $5 \leq count(V^*) \leq 10$ y, a su vez, V^* no debe ser sub-token de ningún otro token

LoRA

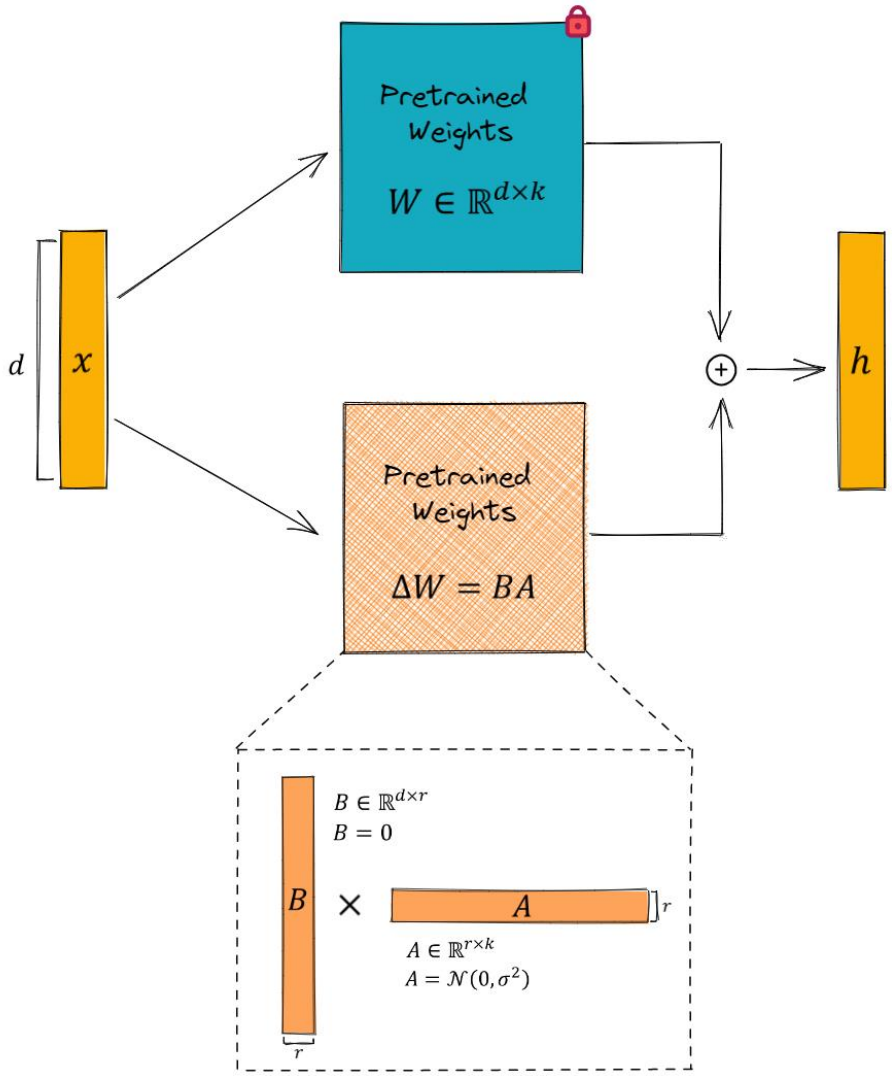
¿Qué es exactamente LoRA?

LoRA, acrónimo de "**Lo**w-**R**ank **A**daptation", es una técnica de *finetuning* que, aunque está diseñada para todo tipo de modelos, en el artículo la proponen para los *"Large Language Models"* (LLM).

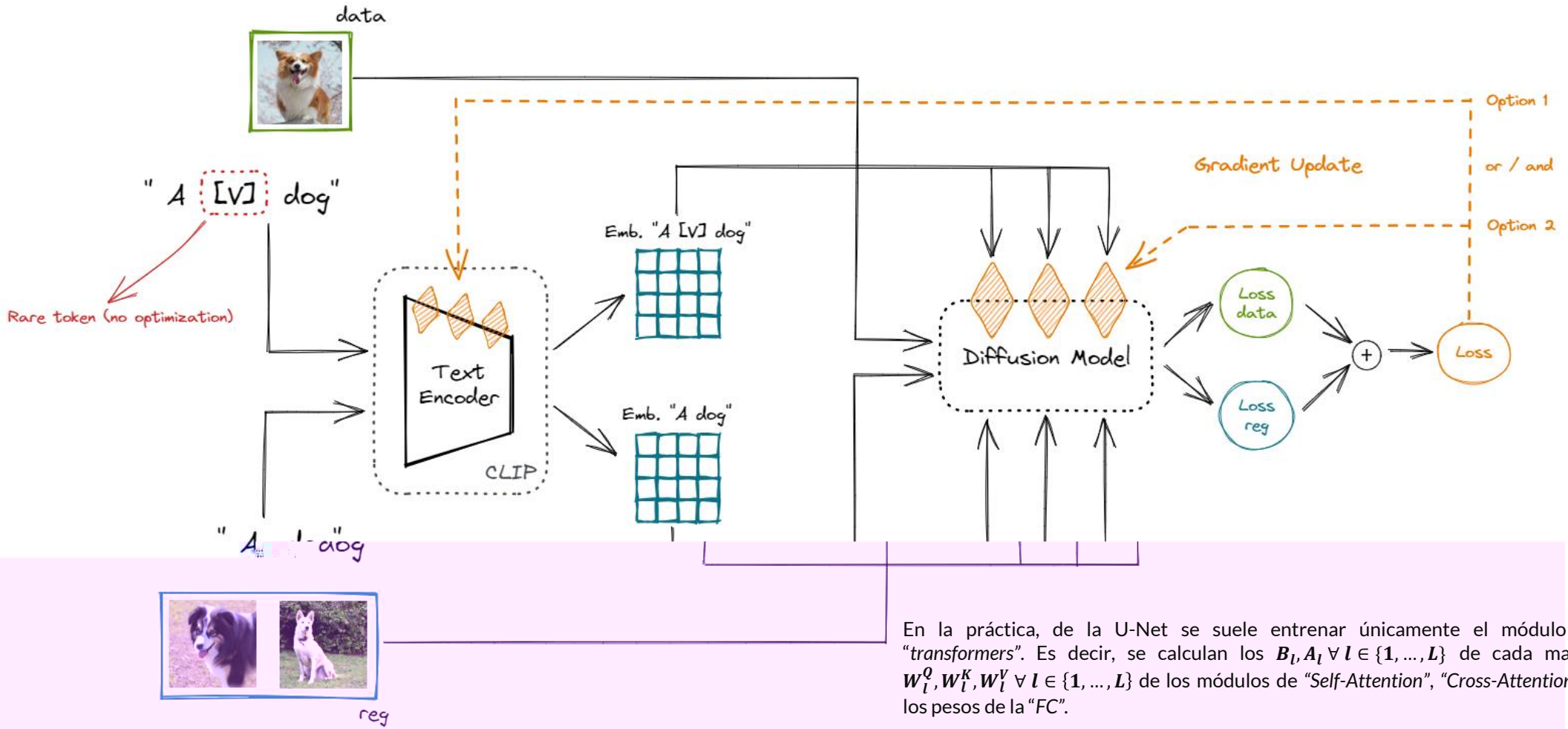
La base de esta técnica reside en la descomposición de la nueva matriz de pesos ΔW en dos sub-matrices B y A que tienen un rango r mucho menor. A la hora de entrenar, únicamente se aprenden los valores de B y A, haciendo que aplicar nuevos pesos sea menos costoso en tiempo y memoria.

$$h = W'x = Wx + \Delta Wx = Wx + BAx$$

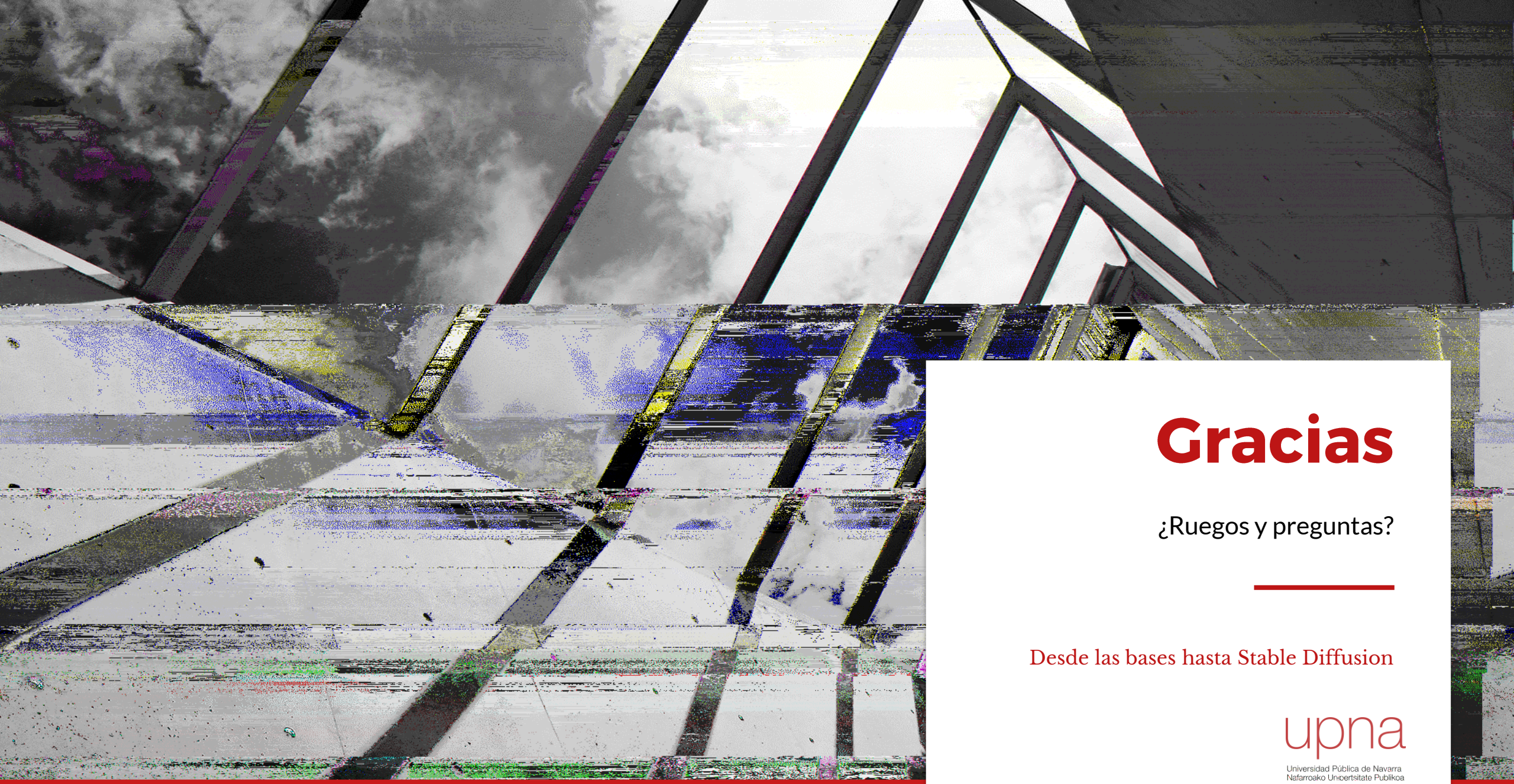
$$W \in \mathbb{R}^{d\times k}, B \in \mathbb{R}^{d\times r}, A \in \mathbb{R}^{r\times k}, r \ll \min(d,k)$$



Dreambooth con LoRA



En la práctica, de la U-Net se suele entrenar únicamente el módulo de "*transformers*". Es decir, se calculan los $\boldsymbol{B_l}, \boldsymbol{A_l}\ \forall\ \boldsymbol{l} \in \{\mathbf{1}, \ldots, \boldsymbol{L}\}$ de cada matriz $\boldsymbol{W_l^Q}, \boldsymbol{W_l^K}, \boldsymbol{W_l^V}\ \forall\ \boldsymbol{l} \in \{\mathbf{1}, \ldots, \boldsymbol{L}\}$ de los módulos de *"Self-Attention"*, *"Cross-Attention"*, y los pesos de la "*FC*".



Gracias

¿Ruegos y preguntas?

Desde las bases hasta Stable Diffusion

upna

Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa